American University of Sharjah
School of Engineering
Computer Science & Engineering Dpt.
P. O. Box 26666
Sharjah, UAE

Instructor: Dr. Michel Pasquier
Office: ESB-2083
Phone: 971-6-515-2883
E-mail: mpasquier@aus.edu
Semester: Spring 2026

CMP 333 ARTIFICAL INTELLIGENCE

PROJECT #1 – SEARCH

Due date/time: Wednesday 25 March 2026, 11 pm

Instructions: Submit to *iLearn,* before or on the due date, a single *Zip/Rar* archive that contains all your *code* files and output *results,* as described hereafter. Make sure to include the *name and ID* of all team members inside all your files and in the name of the archive itself, as per the following format: "CMP333-PR1-MPasquier75321-IZualkernan-70594AAli72571.zip". Follow any further instructions as may be posted on *iLearn.* Late submissions will be penalized as per the course policy.

Note that you are to complete this assignment *as a team of three or four students*. Furthermore, each team must *work independently* and hand in their own original answers. You are *not* allowed to discuss or *share* any solution or to *copy* from others or from any sourced material. Plagiarism and cheating will be severely penalized, starting with a zero grade for the assignment. Recall you are bound by the AUS Academic Integrity Code which you signed when joining AUS.

Assignment: In this first AI course project, you are to program and solve several puzzles and problems using *search algorithms* and *heuristics.* Some of the selected problems were formulated earlier in class or homework/quiz assignments, and some are new.

The given Python code provides an implementation of the *General Search Algorithm* and variants, with some sample problem formulations such as for the 8-puzzle and Pacman. Note that the approach followed is exactly that in our AIMA textbook.

Why in Python? Firstly, by now all of you know Python to some extent. Secondly, this is *not* CMP 321 and the language is *just a tool,* so its usage is very basic. Thirdly, and most importantly, the code reads almost like *pseudo-code,* just like in the book!

Lastly, note that your report should include as always a brief *contribution statement* or table that lists out each team member's role and responsibilities in the project.

Getting ready:

Download the "Project1Search.zip" file from iLearn and unpack it where you wish. The resulting folder contains the following Python files:

"`AI_search.py`" contains a collection of search algorithms i.e., `generalSearch()`, `depthFirstSearch()`, `breadthFirstSearch()`, `iterativeDeepeningSearch()`,

`uniformCostSearch()`, `greedySearch()`, and `astarSearch()`, as well as the supporting data structures i.e., `Stack`, `Queue`, and `PriorityQueue` classes.

Make sure you understand the code. You should not modify the search logic, but later you may need to uncomment and/or modify part of the code to *show the solution* and *performance details,* as elaborated hereafter, and also to add *pruning.* Whatever modification you make, clearly add comments as and where appropriate; each comment line should start with the two `#!` characters (for easy identification).

"`AI_heuristics.py`" contains some examples of *heuristic functions* typical of the 8-puzzle i.e., `hammingDistance()` and `manhattanDistance()`. You will need to modify these and/or add *other heuristic functions* later, in order to solve each specific problem in Part 2. Again, whatever you do, clearly add comments as stated above.

"`AI_problem.py`" contains an *abstract* `SearchProblem` class, with the following methods: `__init__()` that defines the attributes each state consists of; `getStartState()` that returns the initial state of the problem; `isGoalState()` that returns true if a state is a goal, and false otherwise; finally, `getSuccessors()` that returns all the states that can be reached from a given state. This last function will implement the available actions, with their preconditions and effects.

"`EightPuzzleProblem.py`" and "`PacmanProblem.py`" contain *concrete classes* by the same name that extend the `SearchProblem` class and override all four methods above. You should study the code to understand well the problem formulations and how they are implemented i.e., states, actions, cost/s, and heuristic functions. (Note that the famous Pacman game is very similar to the grid world example from class.)

"`AI_solve.py`" contains the `solve()` function that takes as arguments a *problem* (an instance of a concrete subclass of `SearchProblem`) and a *list of search algorithms* (as available from "AI_search.py"), and applies each algorithm in sequence to solve the given problem. By default, the function prints: 1. the name of the problem, 2. the name of the search algorithm employed, 3. the name of the heuristics used (if any), and 4. the details of the solution found or else "no solution".

Details include the cost of the solution (e.g., number of steps or distance), the number of nodes expanded (or number of iterations) and the number of nodes generated (in the search tree). Feel free to modify the code to print other information, change the format, etc. as you see fit. Make sure you always comment your edits, as stipulated earlier. Note that the printed solution path should be *understandable* (!) i.e., showing meaningful state information and not just abstract names or numbers.

<u>8-puzzle and Pacman</u>:

You need to solve *only one* of the following two problems. First, take the sum of the team members' AUS IDs. If that sum is even, apply the `solve()` function to the 8-puzzle ("`EightPuzzleProblem.py`") shown hereafter on the left, then to a another, non-trivial 8-puzzle of your choosing. If the sum is odd, solve the simplified Pacman game ("`PacmanProblem.py`") shown hereafter on the right, then another one using a different, non-trivial Pacman map of your choosing.

For the 8-puzzle, be careful to pick a solvable problem – because half of random-generated 8-puzzles cannot be solved. If you are in that case (you will know because no search algorithm will work), just flip two tiles and you should have a solvable problem. For Pacman, there is no such issue since you're editing the map yourself.

```
puzzle = [1,3,8,
          4,0,2,
          5,7,6]
```

```
pacmap = ["P---------",
          "%%-%%-%-%%",
          "---%--%---",
          "-%%%-%%%%-",
          "----%%-.--",
          "-%%----%%"]
```

Your submission (Word document) must include your *ID sum* calculation, the *output of your program* as plain text (no screenshot) with *all details* as explained earlier, then a *summary table* similar to the below, that reports the results of *all applicable algorithms.* If you find that one algorithm is not applicable, you should *not* use it(!) but instead explain *why* that is the case. Lastly, you should provide a *comparative analysis* of the performance obtained using the search algorithms and heuristics.

Search Algorithm	Nodes Expanded	Nodes Generated	Solution Cost
Depth First Search	597	1065	581
Breadth First Search	112455	134446	23
Iterative Deepening Search	2180357	2180365	23
Uniform Cost Search	112455	134446	23
Greedy Search w/ Manhattan Distance	77	136	43
Greedy Search w/ Hamming Distance	350	578	61
A* w/ Manhattan Distance	244	392	23
A* w/ Hamming Distance	12373	18942	23

Search with Pruning:

Note that the provided `generalSearch()` implementation does not include any pruning. You should now *add parent pruning* to it, and even *ancestor pruning,* as it is quite easy to do so. Adding *full pruning,* however, requires more work (esp. when varying costs are involved) and will thus be treated as *bonus.* Moreover, anyone who implements pruning should allow the user to decide which one they want e.g., by providing an extra argument to the search function: pruning = 'none', or 'parent', or 'ancestor', or 'full'. Note again that only `generalSearch()` should be modified.

Run again the same experiments as in the previous section, but this time applying the *most efficient pruning* technique you implemented. Include a similar *results table,* indicating clearly which type of pruning is used. Lastly, provide likewise a brief *comparative analysis* of the performance obtained with and without pruning.

Stone puzzle, Farmer problem, TSP, Magic triangle:

You need to solve *only one* of the following four problems, for which you have a formulation (from Homework 2). If the sum of the team's IDs ends with 0-2, solve the stone puzzle; if the sum ends with 3-4, solve the farmer problem; if it ends with 5-6, solve the TSP; and if it ends with 7-9, solve the magic triangle. Create a file thus named either "`StonePuzzleProblem.py`", or "`FarmerPuzzleProblem.py`", or "`TravellingSalesmanProblem.py`", or "`MagicTriangleProblem.py`", to model appropriately your problem as a concrete subclass of `SearchProblem`.

Amend then apply your `solve()` function to the problem. Include in your report your *ID sum* calculation, the *detailed results* obtained, the *summary table,* and *comparative analysis,* similarly to what you did with the sample problem in the previous section. Like before, make sure to include all applicable algorithms, to comment on those which are not, and to *define adequate cost and heuristic functions*.

Sokoban sliding puzzle/game:

Sokoban (倉庫番) is a classic sliding puzzle in which the player pushes boxes around in a warehouse, trying to get them to designated storage locations. Note that the game is a nice "microworld" for AI, since it is easy to define but difficult to master, and clearly an abstraction of many real-world planning problems.

Your task is to *devise then implement a formulation* of the Sokoban problem, then *solve some sample puzzles,* as elaborated hereafter. Various boards are provided for testing ("Sokoban_boards.zip"), that offer different challenges. Below are three examples that are of easy, moderate, and hard difficulty (from left to right).

```
#######          ######          ####
# .   #          #@   ##         #  ####
# # # #          # $$  #         # . . #
# # # #          # #. .#         # $$#@#
# $@  #          #     #         ##    #
#     #          #######          ######
#######
```

The rules are as follows: The game is played on a board of squares. Each square is a floor (' ') or a wall ('#'); some squares contain boxes ('$') and some are marked as storage locations ('.'). The player ('@') moves horizontally or vertically (but not diagonally) onto empty squares (never through walls or boxes). The player can move a box by walking next to it and pushing it into the square beyond. Boxes cannot be pulled, and they cannot be pushed into squares that contain walls or other boxes. The number of boxes equals the number of storage locations. The puzzle is solved when all boxes are placed at the storage locations (it does not matter which is where).

Create a file called "`SokobanPuzzleProblem.py`", which models appropriately the Sokoban puzzle as a concrete subclass of `SearchProblem`, with an adequate *cost function.* You should also devise and include in your model a *heuristic function,* to guide the search. The board should be provided as a parameter i.e., a list of strings.

Amend then apply your `solve()` function again to solve *three sample puzzles:* one easy, one moderate, and one hard. Specifically, if the sum of the team's IDs ends with 0-3, solve puzzles #11, #21, and #31; if the sum ends with 4-6, solve puzzles #12, #22, and #32 instead; lastly, if it ends with 7-9, solve puzzles #13, #23, and #33.

For this custom problem, you should apply *heuristic searches* only, using the best *pruning* approach which you devised earlier. Make sure to comment in detail on the *results obtained* with the sample puzzles assigned to you, and the *performance* of the search algorithms, including the impact of the *heuristic function.* (For the latter, it may be easier to compare performance with and without heuristics…)

Submit accordingly your complete *source code* file, show your *ID sum* calculation, the *detailed results* obtained, then your *performance analysis,* as elaborated above.

Note also that your report should include a brief *contribution statement* or table that lists out each team member's role and responsibilities in the project.

Last but not least, don't forget to… have fun solving the puzzles! ☺